# Automated Detection of Pork-Rasher Packaging and Placement Defects

**MMA3001 Numerical Methods and Machine Learning — Individual Project Report** Author: Yijing · Repository: `rasher-qc` · October 2026

> Every number in this report is copied from `results/<dataset>/SUMMARY.md` (generated by `scripts/06_summary.py`) or `results/<dataset>/tables/operating_points.json` (`scripts/07_operating_points.py`). Real-data models were trained on an RTX 4070 laptop GPU (Intel Core, 20 logical CPUs, Windows 11); the synthetic run used a 2-core cloud Xeon. `-block` = leakage-aware split, `-reduced` = predefined split (Section 2.2).

---

## 1. Engineering problem

**Context.** Sliced bacon is packed by laying several rashers in a tray and heat-sealing a lidding film onto the tray flange. Faults arise at both stages: rashers end up *loose* (out of position, overhanging) or *twisted*; the film is *unsealed* along part of the flange, *wrinkled*, or otherwise damaged (*packaging-error*). An unsealed pack leaks, loses its protective atmosphere and spoils before its use-by date; an untidy pack is rejected by the retailer. Inspection is still largely manual — one person watching trays pass at around one per second — which is tiring, inconsistent and leaves no record.

**Problem addressed (narrowed scope).** From one top-view RGB image of a sealed tray on the conveyor: 1. **detect and localise** each defect of the five annotated types (box, class, confidence); 2. **decide** whether the tray should PASS, FAIL, or go to a person (REVIEW); 3. **establish** whether this is accurate *and* fast enough for in-line use, and how far the accuracy figures can be trusted.

**Why it matters.** Missing an unsealed pack is a food-safety and recall risk; failing a good pack costs a re-check. The method is therefore judged mainly on how many defective trays it catches at an acceptable false-reject rate, not only on a detection score.

**Intended use.** Decision support at the end of the packing line: clearly good trays pass, clearly defective trays are diverted, uncertain ones are routed to an operator, and every tray gets a machine-readable record (`results.json`) for traceability. It is not intended to replace the operator for defect types that the data cannot validate (Section 5).

**Limits of the scope.** Only the five annotated defect types and the camera set-up of the dataset (fixed overhead camera, dark textured belt, clear trays, 720 × 540 images). No seal-strength or leak measurement (a seal that looks intact can still leak), no weight or slice-count checks, no hardware integration, and no claims about other products, tray types or lighting.

## 2. Inputs, outputs and data

### 2.1 Top-level inputs and outputs

| | Item | Format, units, domain | Meaning and use |
|---|---|---|---|
| In | Tray image | RGB, uint8 (0–255), 720 × 540 px; accepted range 240–8192 px per side; one tray, top view | The only input at run time |
| In | Training data | YOLO export (`data.yaml`; `labels/*.txt` = class, centre x, centre y, width, height, normalised 0–1) or COCO JSON (pixels) | Training and evaluation only |
| Out | Decision | `PASS`, `REVIEW`, `FAIL`, `UNSUPPORTED_INPUT` | Action on the line |
| Out | Tray score | 0–1: highest confidence among detected defects | Ranking; threshold setting |
| Out | Defect list | class; confidence 0–1; box | Shows the operator where to |

| | Item | Format, units, domain | Meaning and use |
|---|---|---|---|
| | | `x1, y1, x2, y2` in pixels (origin top-left); box area as a fraction of the image | look; rough severity ranking |
| Out | Run report | `results.json`, annotated JPEGs, `summary.html` | Traceability and a supervisor overview |

**Invalid, missing or unsupported inputs** (`rasherqc.io.validate_image`). Missing or undecodable files, greyscale images (colour is part of the evidence), RGB images whose three channels are identical, images too small to resolve a crease (side < 240 px), and grossly under- or over-exposed images (mean grey level outside 20–235, or more than 25 % of pixels clipped) are reported as `UNSUPPORTED_INPUT` with the reason. They are never passed automatically. RGBA images are accepted with the alpha channel dropped, and a warning is recorded. When training data is loaded, unknown class ids or malformed label lines raise an explicit error rather than being skipped. The inputs and outputs of individual functions are documented in Google-style docstrings and in the generated HTML API documentation (`docs/api/index.html`).

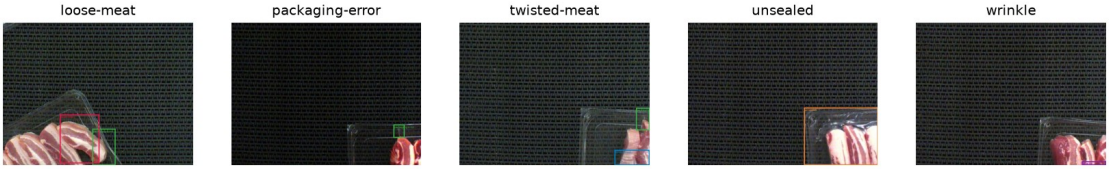
## 2.2 The provided dataset — and what the audit found

*Pork Rasher Error (Packaging)* (Roboflow Universe, version 4, CC BY 4.0). It contains 3,549 images at 720 × 540: 3,349 training images (three exposure-augmented copies, ±10 %, of 1,085 source frames), 120 validation and 80 test images, with boxes for five classes. Before modelling, `scripts/01_data_audit.py` and `scripts/00_prepare_real.py` audited it. Three findings changed the project:

1. **The images are video frames, and the predefined split leaks.** File names keep the source frame number (`frame_with_red_<n>`). For each validation and test image the distance to the nearest training frame was computed: the median is **2.5 frames for test and 3 for validation**, 57 % of test images are within 3 frames of a training image, all are within 30, and 5 % share the exact frame number (they differ only by the exposure augmentation). A test tray is therefore usually the *same physical tray* seen a fraction of a second after a training image, so scores on the predefined test split will be optimistic. A perceptual-hash check confirmed 143 valid/test images that are near-duplicates of training images.
2. **Classes are heavily imbalanced.** In training there are 1,712 `packaging-error`, 1,055 `unsealed`, 72 `twisted-meat`, 66 `loose-meat` and 18 `wrinkle` boxes (all three augmented copies counted). The validation and test splits contain no `wrinkle` boxes at all and only 3–4 `loose-meat` and `twisted-meat` boxes each. Per-class accuracy can only be estimated for `packaging-error` and `unsealed`. A class is reported as *evaluable* only with at least 10 test boxes; below that, a single box changes AP by 10 points or more.
3. **Few good trays to test against.** Only 12 of the 80 test images contain no defect, so a false-reject rate measured on the test split has a wide confidence interval.

**Response.** Two dataset variants are built automatically (`rasherqc.dataset`): * **Predefined split (`-reduced`)**: the given split, keeping one augmented copy per training frame. The detector applies its own, wider colour augmentation, so the copies mostly triple training time. * **Leakage-aware block split (`-block`)**: all source frames are cut into blocks of 300 consecutive frame numbers; whole blocks go to test or validation, and training frames within 60 frames of any held-out frame are removed. The result is 872 training, 155 validation and 173 test frames, with 72 training frames removed by the guard band. The block split also happens to contain more test boxes for the two evaluable classes (98 `packaging-error`, 51 `unsealed`) and more good trays (39), so its estimates are less noisy.

Comparing the two splits measures how optimistic the predefined split is, instead of assuming it (Section 5.4).




*Examples of each class (training split)*

### 2.3 Synthetic verification dataset

A procedural generator (`rasherqc.synthetic`) renders tray images in the same YOLO layout, with the same class names, and with **exactly known boxes**. This plays the role that manufactured solutions play in numerical methods: it verifies that the code is correct — labels, metrics, thresholds, decision logic — independently of noisy real labels, and it isolates single factors such as defect size and glare. It renders shingled streaky rashers on a tray with a sealed flange and film reflections, plus each defect type. It says nothing about accuracy on real trays. In fact, the real scenes (clear trays partly in view on a dark textured belt) look quite different, which is itself a reminder of why the synthetic set is used only for verification.

## 3. Computational solution

### 3.1 Pipeline

```
image → validation (io) → YOLOv8n detector → per-class boxes + confidences
→ tray score s = max defect confidence → decision (PASS / REVIEW / FAIL) →
report
```

### 3.2 Main method: YOLOv8n object detector with transfer learning

**How it works.** YOLOv8 is a one-stage, anchor-free convolutional detector. A CSP-Darknet backbone with C2f blocks extracts feature maps at strides 8, 16 and 32. A path-aggregation neck fuses them, so small defects (a lifted film corner) and large ones (a loose rasher) are both represented. A decoupled head predicts, for every cell of each feature map, the class probabilities and the distances from the cell to the four box edges (each as a discrete probability distribution). Training minimises the weighted sum of three losses: binary cross-entropy for the classes, CIoU for box overlap, and distribution focal loss for the edge distributions. Ground-truth boxes are assigned to cells by a task-aligned assigner. The loss is minimised by mini-batch stochastic gradient descent with the AdamW optimiser, and the model is stored at the epoch with the best validation fitness (0.1·mAP@0.5 + 0.9·mAP@0.5:0.95). At inference, non-maximum suppression (IoU 0.6) removes duplicate boxes.

**Why it was selected.** * **It localises.** The brief asks to detect *and localise*, and an operator needs to see where the fault is. * **Speed.** One forward pass per image suits a line running at about one tray per second. The nano variant has 3.0 M parameters, so it can run on a CPU (Section 6). * **Little data.** Starting from COCO-pretrained weights reuses generic edge, texture and shape features, which matters with only 1,085 distinct training frames. This is tested directly against training from scratch (Section 4). * **The data matches.** The dataset is labelled with boxes, so no re-annotation is needed. * **Reproducibility.** A widely used, documented library; the same code runs on CPU and GPU.

**Assumptions.** 1. Defects are visible in a single top-view RGB frame. 2. Box labels are consistent enough to learn from. 3. The training, validation and test images come from the same process: same camera, belt and trays. 4. A tray's status can be summarised by its most confident defect.

**Settings** (`rasherqc.config`, `rasherqc.detector.train`). Input 416 px on the long side; 60 epochs on GPU (20 on the CPU development run), batch 16, AdamW with the learning rate chosen automatically (≈$1.1 \times 10^{-3}$), early-stopping patience 15, seed 42 with `deterministic=True`. Augmentation: mosaic (switched off for the last 5 epochs), horizontal and vertical flips (a top-view camera has no preferred orientation), and HSV jitter. Rotation is disabled, because the trays always run along the belt.

**Implementation.** `rasherqc.detector` wraps Ultralytics 8.4. Our wrapper fixes one silent pitfall: Ultralytics expects OpenCV's BGR channel order, while the rest of the code uses RGB. A unit test checks that passing an image as an array or as a file gives identical detections.

### 3.3 Tray decision

With tray score $s$: **FAIL** if $s \geq t_{fail}$; **REVIEW** if $t_{low} \leq s < t_{fail}$; otherwise **PASS**. Both thresholds are chosen on the **validation** split. $t_{fail}$ is the highest threshold that still catches at least 95 % of defective validation trays. $t_{low}$ is the highest that catches at least 99 %, so trays in between go to a person. The targets express the unequal costs of the two errors (Section 1). They are engineering choices to be agreed with the plant, not results.

### 3.4 Known limitations of the method

- Boxes are axis-aligned, so box area overstates the true defect area. It is useful for ranking severity, not for measuring it.
- The model cannot detect defect types it has (almost) never seen: there are 6 `wrinkle` training boxes in the reduced set.

- Confidence scores are not calibrated probabilities. Thresholds must be re-chosen if the camera, lighting or product changes.
- Ultralytics is licensed under AGPL-3.0. A closed-source commercial deployment would need a commercial licence or a different detector.

## 4. Alternative solutions

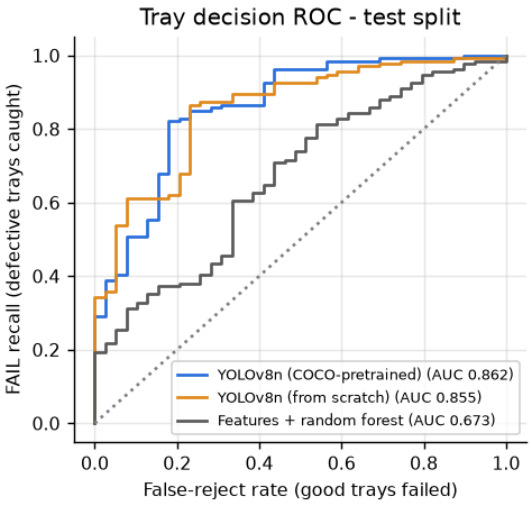
| | Method | What it tests | Localises? |
|---|---|---|---|
| A | **YOLOv8n, COCO-pretrained** (main) | — | yes |
| B | **YOLOv8n trained from scratch**: same architecture, data, settings and seed; random initial weights | whether transfer learning is worth it at this data size | yes |
| C | **Hand-crafted features + random forest**: 80 colour, texture, morphological and orientation features; 500 trees chosen by 5-fold cross-validation on the training split | whether a classical, interpretable, cheap pipeline is good enough for the PASS/FAIL decision | no (image-level only) |

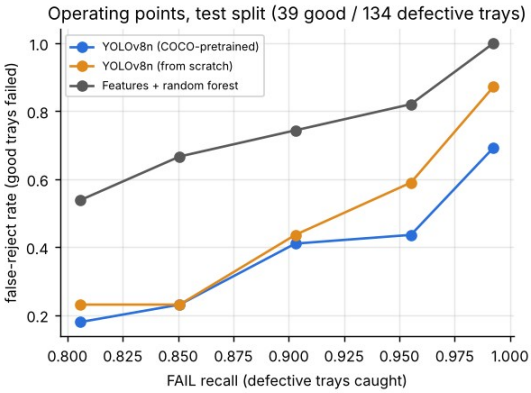
The baseline's features were designed around the physics of each defect. In a correct pack the rashers lie parallel, so edge orientation is coherent; a loose rasher adds a second dominant orientation. Wrinkles and lifted film edges are thin, bright structures, picked up by a white top-hat filter. An unsealed flange shows a dark gap, picked up by a black top-hat filter. A 3 × 3 spatial pyramid pools these responses so that a small local defect is not averaged away. All three methods produce a 0–1 tray score and pass through the *same* threshold procedure (Section 3.3), so their tray decisions are directly comparable.

**Table 1 – Method comparison on the test split.** Main numbers: leakage-aware block split (173 trays: 134 defective, 39 good); in brackets: predefined split (80 trays: 68 / 12). Thresholds chosen on validation (Section 3.3); 95 % bootstrap CIs.

| | A: YOLOv8n pretrained | B: YOLOv8n scratch | C: features + RF |
|---|---|---|---|
| mAP@0.5, evaluable classes | **0.783** (0.589) | 0.654 (0.482) | — |
| mAP@0.5:0.95, evaluable classes | **0.387** (0.317) | 0.314 (0.294) | — |
| AP@0.5 `unsealed` / `packaging-error` | 0.871 / **0.696** (0.851 / 0.328) | 0.827 / 0.481 (0.840 / 0.125) | — |
| Tray ROC AUC | **0.862** [0.78, 0.92] (0.924) | 0.855 [0.79, 0.91] (0.908) | 0.673 [0.57, 0.76] (0.763) |
| False-reject rate at FAIL recall ≈ 0.90 † | **0.41** [0.26, 0.56] (0.08) | 0.44 [0.28, 0.59] (0.08) | 0.74 [0.61, 0.87] (0.50) |
| False-reject rate at FAIL recall ≈ 0.80 † | **0.18** [0.06, 0.31] (0.08) | 0.23 [0.10, 0.37] (0.08) | 0.54 [0.39, 0.69] (0.33) |
| With the validation-chosen thresholds: escape / review / false-reject | 1.5 % / 2.9 % / 56 % | 0.7 % / 3.5 % / 80 % | 0.7 % / 2.9 % / 92 % |
| CPU latency per tray, mean (p95) | 63 ms (80) | ≈ same model size | 385 ms (features 214 ms + RF 165 ms) |
| Training time (RTX 4070) / data needed | 30 min, boxes | 14–35 min, boxes | 3 min on CPU, image labels only |
| Interpretability | boxes show *where*; class + confidence | same | feature importances; no location |

† Descriptive operating points read off the test ROC (`07_operating_points.py`), shown to make the trade-off visible; the decision thresholds themselves are chosen on validation only.




**Selection.** The pretrained detector (A) is selected. It is the best or equal-best on every accuracy measure on both splits; transfer learning matters most for the hard class — `packaging-error` AP 0.70 vs 0.48 (block) and 0.33 vs 0.13 (predefined) — while `unsealed`, a large and high-contrast pattern, is learned almost equally well from scratch. B's training curve was still rising at epoch 60 (best epoch 54) whereas A had plateaued (best epoch 49; 42 with early stopping on the predefined split), so part of B's gap is a matter of training budget; at equal budget, pretraining wins. The classical baseline (C) is **not adequate**: on the block split it has to reject 74 % of good trays to catch 90 % of defective ones, it collapses under every perturbation (Section 6.4), and it cannot say where the defect is. Its feature importances are informative, though: the most useful features were the low percentiles of CIE b* (yellow-blue, i.e. how much exposed tray/film vs meat is visible) and the edge-orientation histogram of the background — the forest is largely detecting *how much of a tray is in the frame*, not the defect itself, which explains its poor transfer to unseen trays. Its one strength is cost: no GPU and image-level labels only.

**What A cannot yet do.** Even A does not meet a demanding operating requirement on unseen trays. Catching ≈ 90 % of defective trays costs ≈ 41 % false rejects; at 80 % recall the false-reject rate drops to ≈ 18 %. The error gallery explains much of this (Section 5.3): false rejects are mostly trays only partly in the frame, where the model detects a plausible `packaging-error` at the film edge that is not annotated.

# 5. Validation

## 5.1 What is validated, and how

| Question | Method | Metric / criterion |
|---|---|---|
| Is the code correct? | Verification tests against hand-calculated cases; synthetic data with exact ground truth; our metric code cross-checked against Ultralytics' validator | Tests pass; our mAP equals the library's to within interpolation differences |
| How well are defects localised? | Withheld test split | AP@0.5 per evaluable class, mAP@0.5, mAP@0.5:0.95 |
| How good is the tray decision? | Thresholds fixed on validation, applied **once** to test; 95 % bootstrap confidence intervals (1,000 resamples of test images) | FAIL recall (share of defective trays caught), false-reject rate, ROC AUC, escape and review rates |
| How optimistic is the predefined split? | Same pipeline on the leakage-aware block split | Difference in the metrics above |
| How robust is it? | Controlled perturbations of test images, thresholds kept fixed | Recall and false-reject rate against perturbation strength (Section 6.4) |

**Why these metrics.** mAP is the standard detection measure. It includes localisation (IoU ≥ 0.5) and ranks predictions at every confidence level. The line, however, acts on trays, so the decision metrics come first: recall of FAIL trays, because escapes are the costly error, and the false-reject rate, because it is the cost of operating the system. Bootstrap intervals

are reported because the test set is small. **Influence on model selection:** validation data chose the best epoch, the thresholds and the baseline's hyper-parameters (by cross-validation on training data). The test split was used once, after every choice had been made.

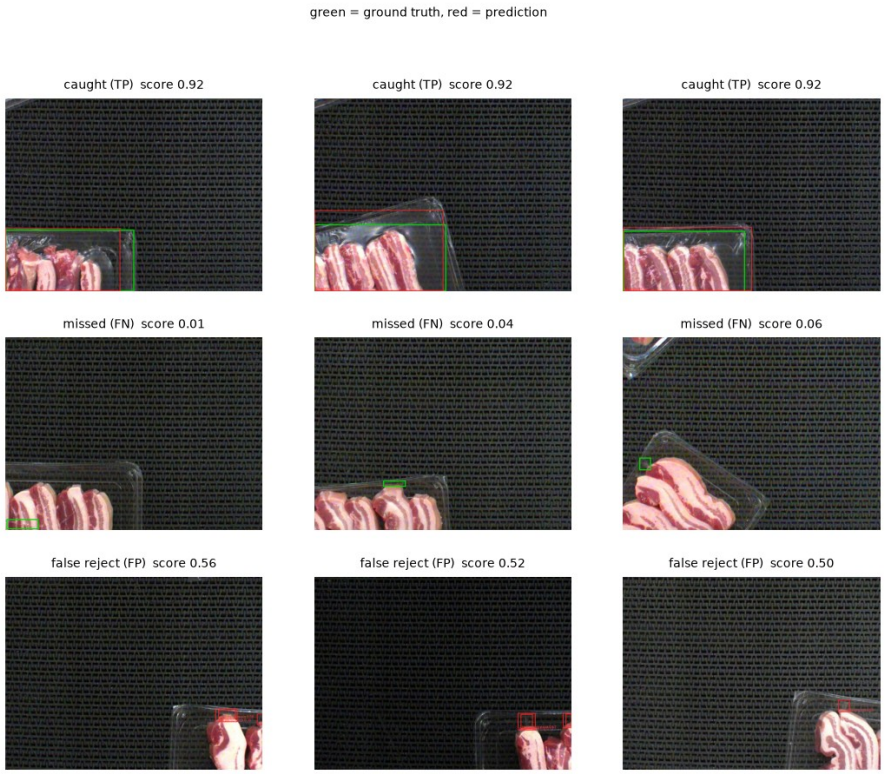
## 5.2 Verification of the implementation

Verification asks whether the code computes what it claims, independently of whether the model is good. Five checks were used (details in Appendix A): **(i)** the metric code against cases worked out by hand and against scikit-learn — this caught a real bug in my first COCO-style AP; **(ii)** our mAP against the independent Ultralytics validator: 0.547 vs 0.542 (block) and 0.344 vs 0.338 (predefined) for the pretrained model; **(iii)** the data loaders by YOLO ↔ COCO round trip and a planted cross-split duplicate; **(iv)** the whole pipeline on the synthetic set with exact labels, where the detector reached mAP@0.5 = 0.978 and a perfect tray ROC — so the much lower real-data scores reflect the problem, not the code; **(v)** every unsupported-input case of Section 2.1. In total **80 tests** pass (`results/test_report.html`) on Linux and on Windows.

## 5.3 Results on the real data

**Detection.** On the block split the pretrained detector reaches AP@0.5 = 0.871 for `unsealed` and 0.696 for `packaging-error` (mAP@0.5:0.95 = 0.387; at a confidence of 0.25, precision/recall are 0.84/0.75 and 0.69/0.68 respectively). Validation mAP on the same classes (0.765) is close to test (0.783), so the choice of best epoch did not overfit validation. The other three classes have one test box each on the block split (3–4 on the predefined split) and are **not evaluated**; for the record, their APs swing between 0 and 1. Detection rate at the operating threshold is 0.80–0.92 in every defect-size bin from < 0.2 % to > 5 % of the image, except 0.5–1 % (0.62, n = 16) (`results/pork-rasher-block/figures/recall_vs_size.png`): small defects are not systematically missed.

**Tray decision.** With thresholds chosen on validation ($t_{fail}$ = 0.079, $t_{low}$ = 0.048) the detector misses 2 of 134 defective test trays (escape rate 1.5 %), sends 2.9 % to review — and fails 56 % of good trays. The thresholds are low because 95 % recall on validation requires them; the full trade-off is in Table 1.




*Error gallery (block split, pretrained detector)*

**Error analysis** (figure above; green = label, red = prediction). *Missed* defective trays carry very small `packaging-error` boxes at the image border or on a tray that is only partly in view. *False rejects* are almost all trays entering the frame from the right: the model reports a `packaging-error` on the visible film edge of a tray whose defect, if any, has not yet entered the frame — or which the annotator did not label in that frame. This points at a modelling limitation

rather than a detector failure: the *frame* is not the *tray*. A tray is seen in several consecutive frames, and the decision should be made once per tray from all of them (Section 7).

### 5.4 How optimistic is the predefined split?

| Pretrained detector, test split | Predefined (leaky) | Block (leakage-aware) |
|---|---|---|
| Tray ROC AUC | 0.924 [0.80, 1.00] | **0.862** [0.78, 0.92] |
| False-reject rate at recall ≈ 0.90 | 0.08 [0.00, 0.29] | **0.41** [0.26, 0.56] |
| mAP@0.5, evaluable classes | 0.589 | 0.783 |
| Good trays in test | 12 | 39 |

The tray-level picture moved in the expected direction: on genuinely unseen trays the decision is clearly worse, and the predefined split's false-reject estimate (1 of 12 good trays) would have badly overstated what the plant could expect. Detection AP did **not** fall — it rose — which I did not expect. Two effects are mixed here and cannot be separated with this data: the block test set has a different composition (98 vs 23 `packaging-error` boxes, mostly in well-populated stretches of video that the model learns well), and the predefined test set is so small that its AP has large uncertainty (± one or two detections on 23 boxes). The honest conclusion is therefore narrower than "the predefined split inflates everything": **the predefined split cannot be used to estimate the false-reject rate**, because it has too few good trays and those it has are near-copies of training frames; the block split, with 3× more good trays from unseen stretches, is the estimate an engineer should use.

### 5.5 What the validation cannot establish

- Performance on `wrinkle`, `loose-meat` and `twisted-meat`. With 0–4 test boxes each, any AP for these classes is anecdotal. The tray decision includes them, but its recall is dominated by `packaging-error` and `unsealed`.
- Performance on other days, shifts, camera positions, lighting, trays or products. All frames appear to come from a single video session. Even the block split only separates moments within one session.
- Whether the labels are correct. Some test boxes look ambiguous (for example, what counts as `packaging-error` versus `unsealed`), and nobody re-annotated them independently.
- Physical seal integrity: the system only ever sees what the camera sees.

## 6. Optimisation and performance

### 6.1 Is it fast enough? (real-time requirement)

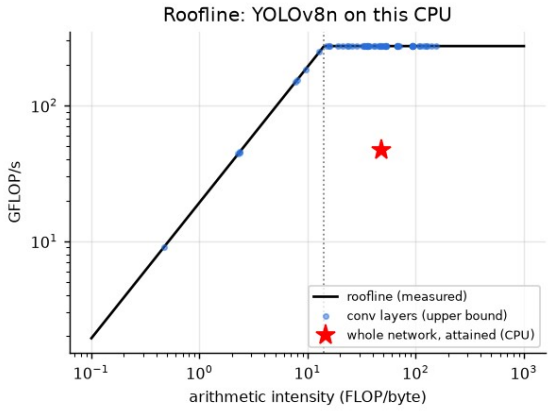
A line packing 60 trays/min leaves **1,000 ms per tray**. All timings are single-image, end-to-end (pre-processing, inference, NMS), measured after warm-up.

| | 256 px | 320 px | **416 px** | 512 px | 640 px |
|---|---|---|---|---|---|
| Laptop CPU, mean / p95 (ms) | 42 / 50 | 48 / 55 | **63 / 80** | 81 / 97 | 103 / 126 |
| Laptop GPU, mean (ms) | 6.3 | 8.2 | **9.2** | 7.3 | 28.9 |
| GFLOPs per image (our counter) | 1.04 | 1.73 | **2.81** | 4.15 | 6.48 |
| mAP@0.5 evaluable (block test) | 0.728 | 0.749 | **0.783** | 0.741 | 0.691 |

**Yes, on a CPU alone, with a margin of about 12× at p95.** A GPU is not needed for this line rate. The classical baseline (385 ms/tray) would also fit, but is not accurate enough. 416 px is selected: it is the most accurate resolution, and 640 px is both 1.7× slower and *less* accurate — most likely because the model was trained at 416 px and objects then appear at an unfamiliar scale. GPU timings at batch 1 are dominated by fixed per-call overhead rather than arithmetic and were erratic between runs on the laptop (≈ 9 ms in the final run, 30–40 ms in an earlier one under different power state), so CPU figures are the dependable basis for the requirement. Batching raises GPU throughput from 25 to 94 images/s (batch 1 → 16) — useful for offline re-inspection, irrelevant on a line that delivers one tray at a time. Peak process memory was 1.8 GB (dominated by the PyTorch/CUDA runtime; the weights are 6.5 MB).

## 6.2 FLOPs, arithmetic intensity and the roofline

The FLOP count was computed layer by layer with forward hooks: for each convolution, FLOPs = 2·Cout·Hout·Wout·(Cin/groups)·kh·kw, and bytes = 4·(|input| + |weights| + |output|) for float32. The counter was verified by a hand calculation in a unit test and agrees with the independent `thop` counter (2.81 vs 2.85 GFLOPs at 416 × 320; the difference is non-convolution layers that `thop` also counts). The 64 convolutions have an arithmetic intensity between 0.5 and 153 FLOP/byte, 48 FLOP/byte for the network as a whole.



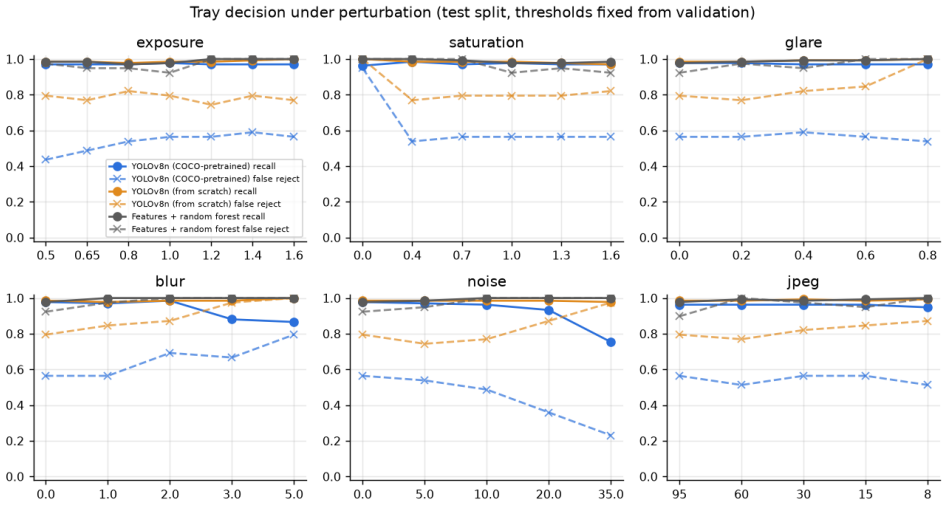

*Roofline*

Measured on the laptop CPU, the attainable compute rate (large matrix multiply) is 274 GFLOP/s and the copy bandwidth 19 GB/s, so the ridge point is ≈ 14 FLOP/byte. Most layers, and the network as a whole, lie to its right: the roofline says the model is **compute-bound** on this CPU and could at best run at ≈ 274 GFLOP/s, i.e. ≈ 10 ms per image. It actually attains **47 GFLOP/s (17 % of peak; 59 ms inference)**. The gap is therefore not memory bandwidth but the efficiency of many small convolutions (short inner loops, im2col/re-layout overhead, thread synchronisation per layer) — consistent with the thread-scaling measurement, where one thread (55 ms) was faster than 14 (66 ms) on this hybrid P-/E-core CPU. Credible next steps are therefore operator-level, not algorithmic: an inference runtime that fuses layers (ONNX Runtime / OpenVINO), or int8 quantisation (≈ 4× fewer bytes and faster integer kernels). The same hardware micro-benchmarks gave 97 GFLOP/s and 4 GB/s in an earlier run on the same laptop — a reminder that laptop power management makes single timing runs unreliable, and that the requirement margin, not the third digit, is what matters.

## 6.3 Optimising the baseline (profiling)

Profiling (`cProfile`) showed four functions taking 64 % of the baseline's time; sharing colour conversions, a smaller morphological footprint and a 32-level GLCM made feature extraction **1.9× faster (409 → 214 ms per image)**, reproducibly on both machines, with a test guaranteeing unchanged features where nothing was meant to change (Appendix B).

## 6.4 Sensitivity and robustness




*Robustness (block split)*

Each perturbation was applied to every test image, with the validation thresholds kept fixed. For the pretrained detector on the block split:

- **Not sensitive:** exposure ×0.5–×1.6 (recall 0.97–0.98, false-reject 0.44–0.59), synthetic glare up to strength 0.8, JPEG down to quality 15. Training-time HSV augmentation covers these.
- **Sensitive:** *blur* — at σ = 3 px recall falls from 0.98 to 0.88 and mAP from 0.55 to 0.22; *loss of colour* — at zero saturation false rejects rise to 95 %; heavy *noise* (σ = 35 grey levels) suppresses detections (recall 0.75). On the predefined split the same pattern held (blur σ = 3 px: false rejects 8 % → 58 %).
- The scratch model and especially the random forest are much more fragile: the forest fails almost every good tray under any perturbation.

Engineering consequence: the camera's **focus and white balance must be monitored** — a defocused or colour-shifted camera silently converts the system into a "fail everything" or "miss things" machine. A cheap guard is to add a focus measure (variance of the Laplacian) and a colour check to the input validation, so that such images become `UNSUPPORTED_INPUT` instead of being scored.

## 7. Limitations and lessons learned

**Principal limitations.**

1. *Data, not the method, limits the conclusions.* Three of five defect classes have 1–4 test boxes; their performance is unknown. Only `unsealed` and `packaging-error` are validated.
2. *One recording session.* Even the block split only separates moments of one video; lighting, belt, trays and product are the same throughout. Performance on another day or line is not established.
3. *Frame ⁄ tray.* Decisions are made per frame, but a tray is seen in many frames and a defect may only be visible in some of them. Most false rejects are trays half in view.
4. *High false-reject rate at high recall* on unseen trays (≈ 41 % at 90 % recall).
5. Labels were not independently checked; box area is only a severity proxy; seal integrity cannot be judged visually.

**Lessons learned.**

- **Audit the data before modelling.** The single most consequential finding of the project — that the predefined split is built from near-identical video frames — came from reading file names, not from any model. Without it the report would have claimed a 8 % false-reject rate instead of ≈ 41–56 %.
- **Validate what the user acts on.** mAP alone would have suggested the block-split model is *better* than the predefined one; the tray-level metrics show the opposite. The decision metric and its uncertainty (bootstrap CIs on 12 vs 39 good trays) are what an engineer can use.
- **Verification first, then validation.** Hand-calculated tests caught a real AP bug, and the synthetic set proved that the pipeline scores perfect labels perfectly, so poor real-data numbers could be attributed to the problem rather than the code. Cross-checking against an independent implementation (Ultralytics) gave agreement within 0.005–0.015 mAP.
- **Measure on the target platform.** Three failures appeared only on Windows; GPU timings varied 4× between runs on the same laptop. Requirements should be judged with margins and percentiles, not single numbers.
- **A simple, transparent baseline earns its place** even when it loses: its feature importances showed what a naive model latches onto (how much tray is in view) and quantified how much the detector actually adds.

**Proposed improvements, in order of expected value.**

1. **Decide per tray, not per frame**: track trays across frames (their motion along the belt is predictable) and fail a tray if a defect is seen consistently in ≥ *k* frames; this directly targets the dominant false-reject mode and can be validated with the existing data by grouping frames.
2. **More and independent data**: record further sessions (other days, lighting) and collect defect classes that are currently missing; re-annotate a sample to measure label noise.
3. **Input guards**: focus (Laplacian variance) and colour checks before detection (Section 6.4).
4. **Model**: fine-tune at the deployment resolution with class-balanced sampling for rare classes; consider a slightly larger model (YOLOv8s), affordable given the 12× latency margin.
5. **Deployment**: export to ONNX/OpenVINO with int8 quantisation, and re-select thresholds on site with the plant's own cost of escapes vs re-checks.

## 8. Reflection on the use of AI

> **Note to myself before submission:** this section must describe my own use of AI honestly and in my own words. The facts below are taken from the project log; edit the evaluation to reflect what I actually checked and understood.

**How AI was used.** Claude (Anthropic), used as an agentic assistant with access to a cloud workspace and, later, to my laptop's project folder, wrote most of the code and tests, proposed the method and validation design, ran the CPU experiments and the synthetic verification, and drafted this report from the generated results. I selected the project, supplied the dataset, ran the GPU pipeline on my RTX 4070 laptop, reported errors back, and reviewed the outputs. The commit history records this split of the work; every commit made by the assistant is labelled as co-authored.

**Where the AI was wrong, and how it was caught.** Eight documented cases (Appendix C) fall into three groups: a numerical bug (the first AP implementation returned 0.0099 instead of 0 — caught by a hand-calculated test); environment assumptions that only failed on my Windows laptop (a Linux-only module, process start-up behaviour, Python-version support of PyTorch and scikit-image); and analysis slips that I caught when reviewing results (a roofline plot that mixed GPU timings with a CPU ceiling, a "parallel" timing that was really serial, over-optimistic run-time estimates). Each was fixed and, where possible, turned into a test.

The most valuable AI contribution was not code generation but the **data audit**: it noticed from the file names that the images are video frames and quantified the leakage of the predefined split, which I would not have checked.

**How the work was verified.** (i) Metric code tested against cases worked out by hand and against scikit-learn; (ii) detection metrics cross-checked against the independent Ultralytics validator; (iii) the full pipeline run on synthetic data with exact labels; (iv) a FLOP counter checked by hand calculation and against `thop`; (v) figures and error galleries inspected visually; (vi) results reproduced on two machines (cloud CPU, laptop GPU).

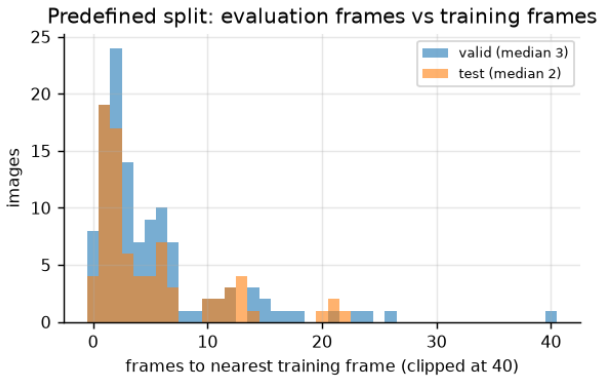
**Evaluation.** AI made it possible to complete a broader and more rigorous project than I could have done alone in the time, and it was consistently strongest at structure, testing discipline and spotting validation problems. Its weaknesses were environment-specific assumptions (operating system, Python versions, hardware) and overconfident estimates — exactly the things that only fail when code meets a real machine. The main risk for me is understanding: code I did not write is code I must be able to explain. To address this, before the presentation I am working through the parts most likely to be questioned — the AP calculation (Section 5.2), the YOLO loss terms and early stopping (Section 3.2), the block split (Section 2.2) and the roofline argument (Section 6) — until I can explain them without notes. *[Edit to state what I actually did.]* If I did the project again I would keep a dated log of prompts and decisions from the start rather than reconstructing it afterwards, and I would run the pipeline on the target operating system from the first day.

## 9. References


1. Jocher, G., Chaurasia, A., Qiu, J. *Ultralytics YOLOv8*, v8.x, 2023–2026. https://github.com/ultralytics/ultralytics (AGPL-3.0).
2. Redmon, J., Divvala, S., Girshick, R., Farhadi, A. "You Only Look Once: Unified, real-time object detection." *CVPR*, 2016.
3. Lin, T.-Y. et al. "Microsoft COCO: Common objects in context." *ECCV*, 2014. (Pretraining data; COCO AP definition.)
4. Everingham, M. et al. "The PASCAL Visual Object Classes (VOC) challenge." *IJCV* 88, 2010. (IoU ≥ 0.5 matching and AP.)
5. Zheng, Z. et al. "Distance-IoU loss: faster and better learning for bounding box regression." *AAAI*, 2020. (CIoU.)
6. Li, X. et al. "Generalized focal loss." *NeurIPS*, 2020. (Distribution focal loss.)
7. Breiman, L. "Random forests." *Machine Learning* 45, 2001.
8. Ojala, T., Pietikäinen, M., Mäenpää, T. "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns." *IEEE TPAMI* 24, 2002.
9. Haralick, R. M., Shanmugam, K., Dinstein, I. "Textural features for image classification." *IEEE TSMC* 3, 1973. (GLCM.)
10. Williams, S., Waterman, A., Patterson, D. "Roofline: an insightful visual performance model for multicore architectures." *CACM* 52(4), 2009.
11. Efron, B., Tibshirani, R. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
12. Kapoor, S., Narayanan, A. "Leakage and the reproducibility crisis in machine-learning-based science." *Patterns* 4(9), 2023.
13. Dataset: "Pork Rasher Error (Packaging)", v4, workspace hello-2aqe0, Roboflow Universe, 2025, CC BY 4.0. https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging
14. Pedregosa, F. et al. "Scikit-learn: Machine learning in Python." *JMLR* 12, 2011; van der Walt, S. et al. "scikit-image: image processing in Python." *PeerJ* 2, 2014.

## Appendix A – Verification details

- **Metrics** (`tests/test_metrics.py`, 21 tests). IoU for identical, half-overlapping (1/3), contained, touching and degenerate boxes; AP against a three-prediction case worked out on paper (TP, FP, TP with 2 ground-truth boxes → 0.5·1 + 0.5·⅔ = 0.833); ROC AUC against scikit-learn including ties; duplicate detections counted as false positives; IoU-threshold dependence. The first COCO-style AP returned 0.0099 instead of 0 for a detector that never matches (a sentinel point leaked into the 101-point interpolation); the test exposed it. The tests also showed that Ultralytics' trapezoidal AP is capped at 0.995 for a perfect detector, part of the small gap to our numbers.
- **Cross-check with Ultralytics** (test split, mAP@0.5): pretrained 0.5467 (ours, COCO rule) / 0.5470 (ours, trapezoid rule) / 0.5420 (Ultralytics) on the block split; 0.3437 / 0.3456 / 0.3378 on the predefined split; scratch 0.2698 / 0.2711 / 0.2629 and 0.2412 / 0.2429 / 0.2561; synthetic 0.978 vs 0.972. Differences come from interpolation details and from classes with 1–4 boxes.
- **Data handling.** Boxes read back from YOLO files equal the rendered boxes to < 0.01 px; YOLO→COCO→load gives identical annotations; a training image planted in the test split is flagged by the dHash leakage check (Hamming distance 0).
- **Synthetic end-to-end run** (CPU, 10 epochs): test mAP@0.5 = 0.978 (classes 0.91–1.00), tray AUC = 1.000, 104/106 defective trays caught, 0/44 good trays rejected; baseline AUC 0.82.
- **Input domain** (`tests/test_io.py`): greyscale, identical channels, too small, under-/over-exposed, clipped, wrong dtype/shape, empty, missing and corrupt files are all rejected with a reason; an end-to-end inspection of {good image, greyscale image, corrupt file} returns {FAIL/PASS, UNSUPPORTED, UNSUPPORTED}.
- **Cross-machine reproducibility.** The evaluable-class mAP@0.5 of the GPU-trained models re-computed on the cloud CPU matches the laptop exactly (0.783 block, 0.589 predefined).



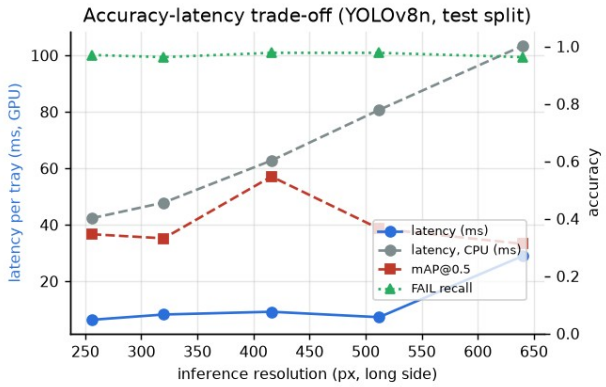

*Distance of evaluation frames to the nearest training frame (predefined split)*

## Appendix B – Performance details

`cProfile` on the reference feature extractor showed four functions taking 64 % of the time: colour conversion to Lab (computed three times), white and black top-hats with a 5-px disc, and the orientation statistics. The optimised version computes each colour conversion once and reuses it, uses a 3-px disc (top-hat cost scales with the disc area) and quantises the GLCM to 32 grey levels ($256^2 \rightarrow 32^2$ matrix). Result: **409 → 214 ms per image (1.9×)**; on the cloud CPU the same change gave 229 → 124 ms (1.85×), so the gain is reproducible across machines. A unit test checks that every feature not deliberately changed is bit-identical between the two versions. Parallelising over images with two processes gave a further 1.6× on Linux; on Windows it falls back to serial (process start-up re-imports the script), which the timing output now flags explicitly.

| | Reference (ms/img) | Optimised (ms/img) |
|---|---|---|
| Colour conversion to Lab | 76.1 | 36.7 |
| White + black top-hat | 121.5 | 18.0 |
| Orientation statistics | 63.9 | 62.3 |
| GLCM | 5.1 | 0.8 |
| **Total (laptop CPU)** | **409** | **214** |

Hardware micro-benchmarks on the laptop (final run): 274 GFLOP/s ($1024^2$ float32 matrix multiply), 17.4 GB/s (NumPy single-thread copy), 19.3 GB/s (PyTorch multi-thread copy). CPU inference vs threads at 416 px: 1 thread 55 ms, 2 threads 101 ms, 4 threads 60 ms, 14 threads 66 ms. GPU throughput vs batch: 25, 42, 57, 94 images/s at batch 1, 4, 8, 16.




*Accuracy-latency trade-off*

## Appendix C – AI-use log

| Problem | How it was caught | Fix |
|---|---|---|
| COCO-style AP returned 0.0099 instead of 0 for a detector that never matches (a sentinel point leaked into the interpolation) | hand-calculated unit test failed | AP computed from observed points only; test kept |
| Peak-memory measurement used `resource`, which exists only on Linux/macOS | pipeline crashed on my Windows laptop | `psutil` used instead |
| Process pools and data-loader workers re-import the script on Windows | anticipated after reading the multiprocessing docs; confirmed by the laptop logs | serial fallback / 0 workers on Windows |
| Set-up script assumed a Python version supported by PyTorch and scikit-image; my default Python 3.15 had no scikit-image build | install failed on my laptop | script selects Python 3.10–3.13 via the `py` launcher |
| Roofline compared GPU timings with a CPU compute ceiling (two different machines in one plot) | spotted when reviewing the first performance results | roofline uses CPU timings only |
| “Parallel” feature-extraction timing on Windows was actually a second serial run | inconsistent timings in the summary | flagged in the output |
| The synthetic tray images were designed before the real images were seen and look quite different | visual comparison with the dataset | synthetic data restricted to code verification |
| Several time estimates were wrong (e.g. “30–60 min” for a run that took about three hours) | observation | — |